Pioneer.jl Flanking-Window Features

Feature reference

2026-06-24

1 Motivation: wide-isolation-window DIA

In data-independent acquisition with *wide* quadrupole isolation windows, a single MS2 window co-isolates many precursors, so a confident identification needs evidence that the signal is real and not bleed-through from a co-isolated neighbor or chromatographic interference. The **flanking-window features** provide this by contrasting two regions of the *same* isolation window:

- the **core**: the contiguous run of MS2 cycles, centered on the precursor's apex scan, where the precursor passes the search threshold (it is PSM-backed by construction — one MS2 scan per cycle per window);
- the **flank**: scans of the same isolation window in the cycles immediately *flanking* the core, where the precursor has *no* passing PSM.

A genuine precursor concentrates MS1 and fragment signal in the core with co-eluting fragments; interference smears into the flank or shows fragments that do not track the precursor.

2 Where they are computed (two pipeline stages)

1. **MainSearch** (`SearchMethods/MainSearch/scoring.jl`) — after best-per-precursor selection, `_mainsearch_flanking_core_bounds!` (`:811`) finds the core window, and the per-precursor loop (`:1045--1097`) writes the intermediate "core" columns into the fold Arrow files: `flanking_core_scan_min`, `flanking_core_scan_max`, `n_contiguous_scans`, `flanking_core_ms1_m0_signal`, `flanking_core_frag_signal`, plus the smoothed apex columns `frag1..8_smoothed_intensity`.
2. **PrecursorScoringSearch** (`PrecursorScoringSearch.jl:199` → `add_wide_window_features_to_fold_fil` in `wide_window_features.jl`) — reads the core columns, gathers the flank scans, extracts MS1 + fragment intensities from the raw spectra, computes the 11 flank feature values in `_wide_flank_feature_values` (`:58`), and writes them back. The two `*_signal` core columns are intermediate and dropped afterward (`:893`).

The final list `FLANKING_WINDOW_FEATURES` (`model_config.jl:49`) feeds the second-pass ScoringSearch LightGBM.

3 Defining the core, and what is raw vs smoothed (MainSearch)

`_mainsearch_flanking_core_bounds!` sorts a precursor's scored rows by `(cycle, scan)`, finds the apex (selected) scan, and walks outward while each neighbor both passes and is in the immediately adjacent cycle (± 1):

```
while lo>1 && pass_mask[prev] && cycle[prev]==cycle[cur]-1; lo-=1; end # left
while hi<len && pass_mask[next] && cycle[next]==cycle[cur]+1; hi+=1; end # right
```

It returns the core scan range `scan_min`/`scan_max`, `n_contiguous_scans`, and the apex/left/right boundary rows.

Core *signals* are summed from RAW intensities. Over the core scans (`scoring.jl:1082--1093`):

$$\text{core\_ms1\_m0\_signal} = \sum_{\text{core scans}} \text{ms1\_m0}, \qquad \text{core\_frag\_signal} = \sum_{\text{core scans}} \sum_{r=1}^{8} \text{frag}_r^{\text{raw}}.$$

Here $\text{frag}_r^{\text{raw}}$ are the matched per-scan intensities (`MAINSEARCH_FRAGMENT_INTENSITY_COLUMNS`); **no smoothing is applied to the sum.**

The smoothed values are a separate, per-fragment APEX height. `_mainsearch_radius1_smooth_fragme` (`:917`) computes, for each of the 8 fragments, a radius-1 weighted average over the apex scan and its two adjacent core scans:

$$\text{frag}_r^{\text{smoothed}} = \frac{2\,a_r + l_r + r_r}{\text{denom}},$$

where a_r, l_r, r_r are the raw apex/left/right intensities and `denom` counts the present neighbors. This is *not* summed — it is a denoised *peak height*. A single-scan raw apex is spiky; the 3-point weighted average is a stabler estimate of the true peak. These `frag*_smoothed_intensity` values are used (i) directly as LightGBM features and (ii) as the core apex in the flanking `frag_apex_gt2x` feature below. *So smoothing benefits the apex/peak-height comparison, not the sum.*

4 Gathering the flank and extracting signal (PrecursorScoringSearch)

- `_wide_window_index_spectra` (`:286`) indexes every MS2 scan by its isolation window key `(center_mz, isolation_width)` — one scan per cycle.
- `_wide_group_flank_window_scans_from_core!` (`:420`) collects the same-window scans in cycles flanking the core (outside [scan_min, scan_max], within `WIDE_WINDOW_CYCLE_MARGIN`= 3 cycles). No flank scans ⇒ zero/default feature values.
- `_wide_top_fragment_idxs` / `_wide_group_fragment_windows` pick the top-8 library fragments and m/z windows; scan-centric extraction (`_wide_fill_scan_centric_{ms1_m0,fragments}!`, batched at 50000 groups) fills the *raw* flank vectors `flank_ms1_m0` and `flank_fragments` (#flank scans × 8).

5 The features (`_wide_flank_feature_values`, line 58)

Let c_{m0}, c_f be the core MS1-m0 / fragment signals (raw sums); a_r the smoothed core apex of fragment r; $\mathbf{m}$ the raw flank MS1-m0 vector (one per flank scan); F the raw flank fragment matrix (scan × fragment). Floor all intensities at 0; let $s_j = \sum_r F_{j,r}$, and $\rho(\cdot,\cdot)$ be Pearson correlation (`_wide_window_pcor`; 0 for < 2 points or zero variance).

`flanking_ms1_m0_candidate_fraction` [Float32]

$\frac{c_{m0}}{c_{m0} + \sum_j \mathbf{m}_j}$ — fraction of the precursor's MS1 monoisotopic (m0) signal sitting in the core vs. spread into flank scans. $\to 1$ for a clean precursor.

`flanking_frag_candidate_fraction` [Float32]

$\frac{c_f}{c_f + \sum_j s_j}$ — the same core-vs-flank fraction for summed fragment signal.

`flanking_ms1_frag_sum_corr` [Float32]

$\rho(\mathbf{m}, \mathbf{s})$ across flank scans — do the MS1 precursor and the fragment sum co-vary off-apex? (co-elution sanity check in the flank).

`flanking_frag_corr_mean` [Float32]

mean of all pairwise $\rho(F_{:,a}, F_{:,b})$ over flank scans, among fragments with signal — average fragment-fragment co-elution in the flank.

`flanking_n_correlated_fragments` [UInt8]

count of fragments whose leave-one-out correlation $\rho\big(F_{:,r},\, \mathbf{s} - F_{:,r}\big) > 0.7$ (`WIDE_WINDOW_CORR_THRESHOLD`) — how many fragments track the rest.

`flanking_n_correlated_fragments_bitvec_rank` [UInt16]

`_bitvec_pattern_rank` of the 8-bit mask of *which* fragments exceeded 0.7 — a learned categorical encoding of the correlated-fragment *pattern*, not just the count.

`flanking_frag_corr_strength` [Float32]

$\sum_r \max(\min(\rho_r, 1), 0)$ over the leave-one-out correlations ρ_r — continuous total correlation strength across the 8 fragments.

`flanking_frag_corr_effective_n` [Float32]

$\frac{\left(\sum_r \rho_r^+\right)^2}{\sum_r (\rho_r^+)^2}$ (participation ratio of the positive leave-one-out correlations) — the effective number of co-eluting fragments.

`flanking_frag_corr_best_m0` [Float32]

$\rho(F_{:,b,\mathbf{m}})$ where b is the fragment with the highest mean pairwise correlation (the "anchor") — does the best fragment track the MS1 precursor across the flank?

`flanking_signal_support` [Float32]

$\frac{\#\{j : \mathbf{m}_j > 0 \text{ or } s_j > 0\}}{\#\text{flank scans}}$ — fraction of flank scans carrying any signal.

`frag_apex_gt2x_flank_bitvec_rank` [UInt16]
`_bitvec_pattern_rank` of the 8-bit mask of fragments whose smoothed *core apex* $a_r \geq 2\times$ their flank average $\bar{F}_{:,r}$ — which fragments genuinely peak at the apex rather than being flat/interference. (This is the one feature that uses the smoothed apex.)

`n_contiguous_scans` [UInt16]
(from the core bounds, not `_wide_flank_feature_values`) the number of contiguous cycles in the core run — how many cycles the precursor persisted.

The 11 values from `_wide_flank_feature_values` plus `n_contiguous_scans` make up the 12-entry `FLANKING_WINDOW_FEATURES`.

6 Helper functions

`_mainsearch_flanking_core_bounds!  (scoring.jl:811)` [MainSearch]
Find the contiguous-cycle core window around the apex; return scan_min/max, n_scans, and the apex/left/right boundary rows.

`_mainsearch_radius1_smooth_fragments!  (scoring.jl:917)` [MainSearch]
Radius-1 weighted apex smoothing $(2a + l + r)$/denom producing `frag*_smoothed_intensity`.

`_wide_window_pcor (:5)` [wide_window_features.jl]
Single-pass Float32 Pearson correlation; 0 if < 2 points or zero variance.

`_wide_candidate_signal_fraction (:32)` [wide_window_features.jl]
core/(core + flank), guarding divide-by-zero.

`_wide_flank_feature_values (:58)` [wide_window_features.jl]
Compute the 11 flank features from the core scalars/apex and the raw flank arrays.

`_wide_window_index_spectra (:286)` [wide_window_features.jl]
Index scans by isolation window `(center_mz, isolation_width)`.

`_wide_group_flank_window_scans_from_core!  (:420)` [wide_window_features.jl]
Collect same-window scans in cycles flanking the core (margin = 3 cycles).

`_wide_fill_scan_centric_{ms1_m0,fragments}!  (:578/:601)` [wide_window_features.jl]
Read raw-spectra MS1 m0 and fragment intensities at each flank scan (scan-centric, batched).

`_bitvec_pattern_rank` [shared]
Map an 8-bit fragment pattern mask to a learned rank (file-specific table).

7 Constants

`WIDE_WINDOW_CYCLE_MARGIN`= 3 (flank reaches up to 3 cycles either side); `WIDE_WINDOW_CORR_THRESHOLD`= 0.7 (correlated-fragment cutoff); `FLANKING_SCAN_CENTRIC_BATCH_SIZE`= 50000 (groups per scan-extraction batch).

8 Consumption

The features are appended to the per-fold second-pass PSM Arrow files and feed the ScoringSearch second-pass LightGBM (`FLANKING_WINDOW_FEATURES`). They are most informative on wide-window instruments (e.g. Sciex SWATH), where co-isolation is heaviest, and near-constant on narrow-window data.